Code Coverage Report

Project: Voice Assistant for Multimedia Controls - Team 3 **Prepared for:** Dhanush S S, Integration Lead **Execution date:** 07 November 2026, 01:50 IST (UTC+05:30) **Application source commit:** `91d5a74d815a2daff04afd896e4f135877d552b9` **Test mode:** author-run automated host tests; independent acceptance pending.

The course testing window shown in the brief is **2-6 November**. These tests were actually run on **7 November 2026 IST**; results are not backdated. UTC timestamps may fall on 6 November. Report date and evidence date are distinct from the course deadline.

Measured results

Scope	Primary code metric	Result
12 portable C++ implementation files	Executed source lines / executable source lines	**662 / 691 = 95.80%**
Same C++ scope	Executed functions	**55 / 61 = 90.16%**
Same C++ scope	Executed coverage regions	**617 / 644 = 95.81%**
Six optional Python prototype files	Executed statements / statements	**50 / 120 = 41.67%**

"Code coverage" here means line coverage for C++, and statement coverage for Python. These are not SRS requirement-acceptance percentages. Results are not averaged across the two different runtimes or presented as 100% project coverage.

Explicit C++ denominator

All **12 existing portable implementation `.cpp` files** under `firmware/lib/` are compiled and included, including files not directly referenced by a suite. Only those `.cpp` mappings enter the percentages. **Headers/inline-only logic** (including `reset_button.h`), `firmware/src` hardware adapters/entry point, examples, tests, generated code, Unity, OpenSSL/framework internals and tooling are excluded. ESP32-only conditional code (notably mbedTLS) is not compiled in this host run. Unimplemented features have no code denominator and are not implicitly "covered". This is not whole-product or target-firmware coverage.

The scope inventory lists compiled files and excluded headers. The BOOT detector has six passing tests but no percentage in this measurement. Report it as tested, not as measured coverage.

C++ file results

Portable implementation file	Covered / executable lines	Line coverage
asr_client/command_gate.cpp	34 / 34	100.00%
asr_client/command_matcher.cpp	139 / 139	100.00%
ble_hid/ble_hid.cpp	57 / 69	82.61%
command_map/command_map.cpp	113 / 115	98.26%
connectivity/authenticated_pairing.cpp	32 / 32	100.00%

Portable implementation file	Covered / executable lines	Line coverage
connectivity/ble_discovery.cpp	24 / 24	100.00%
connectivity/credential_vault.cpp	77 / 78	98.72%
connectivity/factory_reset.cpp	11 / 11	100.00%
connectivity/wifi_provisioning.cpp	52 / 52	100.00%
diagnostics_ota/diagnostics_ota.cpp	48 / 48	100.00%
feedback/feedback.cpp	64 / 78	82.05%
pipeline/command_dispatch.cpp	11 / 11	100.00%

There are **29 uncovered executable C++ lines** in scope. See the annotated evidence for exact locations. BLE stub/default paths and feedback paths account for most uncovered lines. Allocation/crypto-error paths also need targeted fault-injection consideration. This report adds no tests in teammates' modules.

Optional Python prototype - full source scope

Prototype file	Covered / statements	Statement coverage
command_classifier.py	7 / 7	100.00%
command_mapper.py	3 / 3	100.00%
main.py	30 / 34	88.24%
media_controller.py	10 / 25	40.00%
receive_audio.py	0 / 43	0.00%
speech_to_text.py	0 / 8	0.00%

`receive_audio.py` and `speech_to_text.py` are **0%**, not omitted. Serial transport, actual local model loading/transcription and real Windows media execution still need suitable tests/environments. Python coverage's displayed combined figure is **41.03%** because it combines statements and branch outcomes (64/156); that is different from statement-only **41.67%** and is not substituted for it.

No minimum line/statement target was specified for this reporting task. The measured result is reported without inventing a pass threshold. Improvements should target meaningful uncovered behavior, not merely higher percentages.

Scope and acceptance boundary

The approved project uses ESP32-WROOM and offline fixed-vocabulary recognition. No live cloud API, microphone, flash operation or Bluetooth/media action was performed by this report run. Passing host tests is not completion of the physical SRS/STP acceptance criteria. This report does not set RTM A/F statuses, close issues, complete INT-02, create a release tag or freeze development.

The locally revised SRS/SAD/STP/backlog implement Dhanush's offline redesign; some revisions are still unpublished and remote source text remains inconsistent. The exact source-document hashes are in source-baselines.json. Harsh/DOC-01 must reconcile and publish those baselines before final acceptance. Legacy `TC-ASR-*` IDs are retained; their names do not authorize cloud recognition.

Reproduction and evidence

From the repository root, with PlatformIO installed:

```
pio test -d firmware -e native
python3 -m pip install coverage==7.17.2
python3 scripts/measure_coverage.py --output /tmp/vamc-report-rerun
```

The coverage runner currently supports **macOS** with Xcode command-line tools, Homebrew `openssl@3` and Unity installed by the PlatformIO test run. It compiles the same existing C++ tests and library sources with Apple Clang/LLVM 21 at `-O0`, using `-fprofile-instr-generate -fcoverage-mapping`. Test/Unity code is not instrumented. All 12 portable `.cpp` objects are linked into every suite; the 15 raw profiles are merged once, and one binary provides their shared mappings. This avoids counting shared code 15 times. The runner then invokes Python coverage with `--branch`, including all six prototype source files and excluding its tests. It records tool versions, commands, source commit and suite counts.

- Run manifest and exact commands
- Coverage scope inventory
- PlatformIO native test output
- C++ coverage summary
- C++ annotated lines and branch counts
- C++ machine-readable coverage
- Python test output
- Python coverage summary
- Python machine-readable coverage
- Python annotated HTML

Method reference: Clang source-based coverage documentation. Execution evidence, rather than documentation examples, supplies every result here.